GetHere: Food Delivery Platform

BLG 317E - Database Systems Term Project Report

Group Members:
Furkan Yalçın[150220116]
Burak Emre Polat [150230328]
Amr Nael Zuhdi Taweel [150220937]
Yavuz Selim Yağlıca [150220103]
Hasan Yalçın Arıkanoğlu [150220341]

January 2, 2026

Contents

1 Introduction

GetHere is a comprehensive food delivery platform designed to connect customers with restaurants through an efficient courier network. The system provides a complete ecosystem for ordering food, managing restaurants, and coordinating deliveries.

1.1 Project Overview

The platform serves three primary user types:

- **Customers (Users):** Browse restaurants, place orders, track deliveries, and rate services
- **Restaurant Managers:** Manage restaurant profiles, menus, create job positions, and monitor orders
- **Couriers:** Search and apply for positions, manage deliveries, and track performance

1.2 Source Dataset

We utilized the **Zomato Restaurant Database**, available on Kaggle, as the foundational data source for this project. This comprehensive dataset provides a rich, relational structure that includes detailed information across several entities:

- **Users:** Profiles with demographics like age, occupation, and marital status.
- **Restaurants:** Dining establishments with attributes like cuisine, city, rating, and cost.
- **Menu & Food:** Granular details on dishes, prices, and dietary types (Veg/Non-veg).
- **Orders:** Transactional history linking users to restaurants and specific menu items.

This multi-table dataset was ideal for simulating a real-world food delivery platform, allowing us to implement complex features like order history analysis, restaurant recommendations, and dashboard analytics right from the start.

1.3 Technology Stack

1.3.1 Backend Technologies

- **Programming Language:** Python 3.x
- **Web Framework:** Flask (with Blueprints for modular architecture)
- **Database:** MySQL 8.0 (Relational Database Management System)
- **Database Connector:** mysql-connector-python
- **Password Security:** bcrypt for password hashing
- **Session Management:** Flask-Session

1.3.2 Frontend Technologies

- **Markup:** HTML5
- **Styling:** CSS3 with custom designs
- **UI Framework:** Bootstrap 5
- **Icons:** Bootstrap Icons
- **Scripting:** JavaScript (Vanilla JS for AJAX calls)
- **Templating:** Jinja2 (Flask's template engine)

1.3.3 Development Tools

- **Version Control:** Git
- **Database Tool:** MySQL Workbench
- **API Testing:** Postman
- **IDE:** Visual Studio Code

2 Team Responsibilities

The project was divided among team members based on database entities and their associated functionalities:

Table 1: Team Member Responsibilities

Team Member	Primary Responsibility	Modules Developed
Furkan	Courier & Task System	`courier_view.py`, `task_view.py` Position management (shared) Leaderboard, Delivery History
Burak Emre Polat	Restaurant System	`restaurant_view.py` Position creation (shared) Restaurant Dashboard
Amr Nael Zuhdi Taweel	User System	`user_view.py` Authentication, Profile Order History
Yavuz Selim Yağlıca	Order System	`order_view.py` Order CRUD operations Rating System
Hasan	Menu & Food System	`menu_view.py`, `food_view.py` Global Food Catalog Menu Management

2.1 Shared Components

The **Positions** table and functionalities were collaboratively developed by Furkan and Burak Emre Polat:

- **Burak:** Position creation from restaurant manager's perspective
- **Furkan:** Position search, application, and courier enrollment logic

3 Entity-Relationship Diagram

3.1 ER Diagram Overview

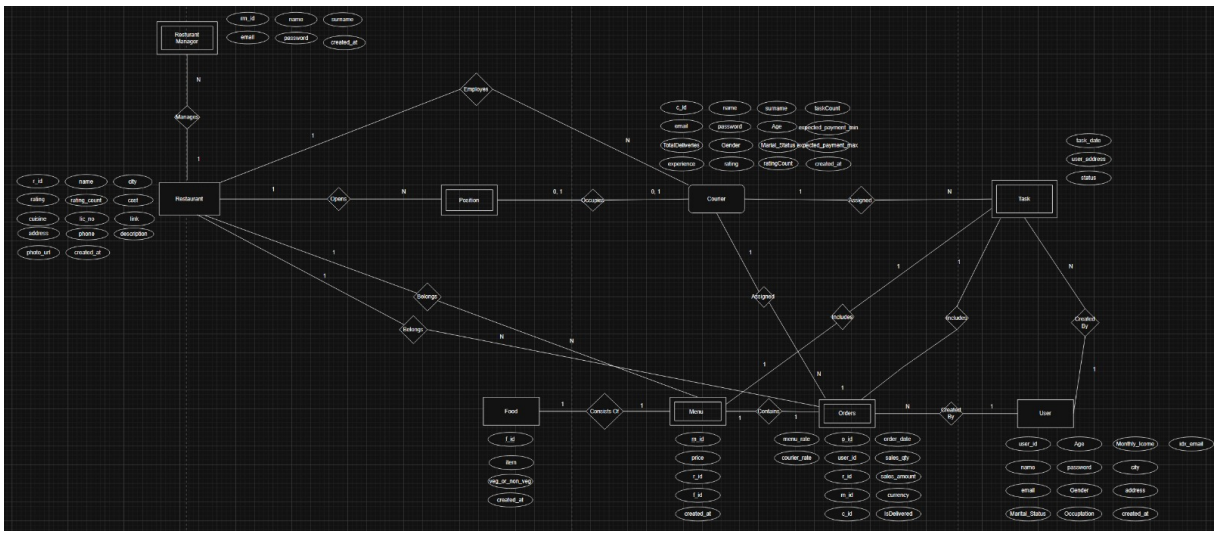


Figure 1: Entity-Relationship Diagram for GetHere Platform

3.2 Domain Description and Relationships

GetHere is a food delivery ecosystem where **Users** (customers) order food from **Restaurants**, and **Couriers** deliver those orders. Each entity plays a specific role in this real-world business process.

3.2.1 Core Business Entities

Users and Ordering Users are customers who browse restaurants, place orders, and rate their experience. A user can place multiple orders over time, but each order belongs to exactly one user. When a user deletes their account, all their order history is removed from the system (CASCADE).

Restaurants and Management A Restaurant represents a food establishment registered on the platform. Each restaurant can have multiple managers who handle day-to-day operations like updating menus, viewing orders, and creating job positions for couriers.

Couriers and Deliveries Couriers are delivery personnel who work for restaurants. A courier can only work for one restaurant at a time but can switch employers. They receive delivery tasks, complete them, and build their rating over time. Couriers must be at least 18 years old and start with a trust rating of 3.0.

Food Catalog The Food entity represents a global catalog of food items (e.g., "Chicken Burger", "Veg Fried Rice"). This allows different restaurants to offer the same food item without duplication, enabling platform-wide search and analytics.

3.2.2 Weak Entities and Dependencies

Restaurant_Manager (Weak Entity) A Restaurant_Manager cannot exist without a Restaurant. Managers are created when a restaurant registers and are automatically deleted when the restaurant is removed from the platform (ON DELETE CASCADE). Multiple managers can manage the same restaurant, allowing for shared administrative responsibilities.

Menu (Weak Entity) A Menu item represents a specific food offering at a specific restaurant with its own price and cuisine category. Menu items depend on both Restaurant and Food entities. If a restaurant closes, all its menu items are deleted. The Menu links the global Food catalog to individual restaurant offerings.

Position (Weak Entity) A Position represents a job listing opened by a restaurant to hire couriers. Positions have requirements (minimum experience, minimum rating) and offer a monthly payment. When a courier applies and is accepted, the position closes and records the `hired_at` timestamp. If the restaurant is deleted, all its positions are removed.

Task (Weak Entity) A Task represents a single delivery assignment. It is the weakest entity in the system, depending on Orders, Courier, User, and Menu. A task is created when an order is placed and assigned to the least busy courier. Tasks track delivery status and are deleted if the associated order, courier, or user is removed.

Orders (Weak Entity) An Order connects a User to a Restaurant through a specific Menu item, assigned to a Courier for delivery. Orders store quantity, amount, ratings for both food and courier, and delivery status. Orders cannot exist without a courier (RESTRICT), ensuring every order has an assigned delivery person.

3.2.3 Key Relationships

- **Restaurant *Opens* Position (1:N):** A restaurant can create multiple job listings to hire couriers.
- **Courier *Occupies* Position (1:0..1):** A courier can occupy at most one position at a time. The position tracks when they were hired (`hired_at`) and how many deliveries they've made.
- **Restaurant *Employs* Courier (1:N):** A restaurant can have multiple couriers working for it, but each courier works for only one restaurant.

- **Restaurant_Manager *Manages* Restaurant (N:1):** Multiple managers can manage one restaurant, enabling team-based administration.
- **Restaurant *Has* Menu (1:N):** A restaurant offers multiple menu items, each linking to the global food catalog.
- **Menu *Consists of* Food (N:1):** One food item can appear on multiple restaurant menus with different prices.
- **User *Creates* Order (1:N):** A user can place multiple orders over time.
- **Order *Assigned To* Courier (N:1):** Each order is assigned to exactly one courier, but a courier handles multiple orders.
- **Order *Includes* Task (1:1):** Each order has exactly one delivery task, and each task belongs to one order.
- **Courier *Assigned to* Task (1:N):** A courier can have multiple active tasks (deliveries in progress).

3.3 Relationship Cardinalities

Table 2: Entity Relationships and Cardinalities

Relationship	Entity 1	Entity 2	Cardinality
Manages	Restaurant_Manager	Restaurant	N:1
Opens	Restaurant	Position	1:N
Occupies	Courier	Position	1:(0,1)
Employs	Restaurant	Courier	1:N
Belongs (Menu)	Restaurant	Menu	1:N
Consists Of	Food	Menu	1:N
Contains	Menu	Orders	1:N
Created By (Order)	User	Orders	1:N
Assigned (Order)	Courier	Orders	1:N
Includes (Task)	Orders	Task	1:1
Assigned (Task)	Courier	Task	1:N
Created By (Task)	User	Task	1:N

4 Database Schema

4.1 Schema Definition

The database uses MySQL with InnoDB engine for transaction support and foreign key constraints.

```
-- Positions Table with hired_at for accurate performance tracking
CREATE TABLE Positions (
    p_id INT PRIMARY KEY AUTO_INCREMENT,
    r_id INT NOT NULL,
    c_id INT,
```

```sql
    city VARCHAR(50),
    req_exp INT DEFAULT 0,
    req_rating DECIMAL(3,1) DEFAULT 0,
    payment DECIMAL(10,2) NOT NULL,
    isOpen BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    hired_at TIMESTAMP NULL DEFAULT NULL,
    deliveries_made INT DEFAULT 0,
    FOREIGN KEY (c_id) REFERENCES Courier(c_id) ON DELETE SET NULL,
    FOREIGN KEY (r_id) REFERENCES Restaurant(r_id) ON DELETE CASCADE
);

-- Task Table (Weak Entity)
CREATE TABLE Task (
    t_id INT PRIMARY KEY AUTO_INCREMENT,
    o_id INT NOT NULL,
    c_id INT NOT NULL,
    user_id INT NOT NULL,
    m_id INT,
    task_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    user_address TEXT,
    status BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (o_id) REFERENCES Orders(o_id) ON DELETE CASCADE,
    FOREIGN KEY (c_id) REFERENCES Courier(c_id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES User(user_id) ON DELETE CASCADE,
    FOREIGN KEY (m_id) REFERENCES Menu(m_id) ON DELETE SET NULL,
    UNIQUE KEY unique_order (o_id)
);
```

Listing 1: Core Schema Definition

5 CRUD Operations

5.1 User Module (Amr Nael Zuhdi Taweel)

5.1.1 Create - User Registration

```python
@user.route('/submit_form', methods=['POST'])
def user_submit_signup_form():
    full_name = f"{request.form.get('first_name')} {request.form.get('last_name')}"
    password_hash = bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt())

    query = """
        INSERT INTO User
        (name, email, password, Age, Gender, Marital_Status,
         Occupation, Monthly_Income, city, address)
        VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
    """
    cursor.execute(query, values)
    db.commit()
```

Listing 2: User Registration

5.1.2 Read - User Profile

```
cursor.execute("SELECT * FROM User WHERE user_id = %s", (user_id,))
user_data = cursor.fetchone()
```

Listing 3: Fetch User Profile

5.1.3 Update - Profile Update

```
cursor.execute("""
    UPDATE User SET name=%s, email=%s, Age=%s, Gender=%s,
    Marital_Status=%s, Occupation=%s, Monthly_Income=%s,
    city=%s, address=%s WHERE user_id=%s
""", (name, email, age, gender, status, occupation, salary, city,
   address, user_id))
```

Listing 4: Update User Information

5.1.4 Delete - Account Deletion

```
cursor.execute("DELETE FROM User WHERE user_id=%s", (user_id,))
db.commit()
```

Listing 5: Delete User Account

5.2 Restaurant Module (Burak Emre Polat)

5.2.1 Create - Restaurant and Restaurant Manager

This handles the creation of a new restaurant entity and its first manager. It performs two INSERT operations within a transaction: one for the Restaurant table and one for the Restaurant_Manager table linked to that restaurant.

```
@restaurant.route('/submit_signup', methods=['POST'])
def restaurant_submit_signup():
    # ... (input validation and file handling) ...
    try:
        # 1. Create Restaurant
        restaurant_query = """
            INSERT INTO Restaurant (name, city, address, cuisine, phone,
    description, photo_url, lic_no, link, rating, rating_count)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, 3.0, 0)
        """
        restaurant_values = (restaurant_name, city, address, cuisine,
   phone, description, photo_filename, lic_no, link)
        cursor.execute(restaurant_query, restaurant_values)
        new_restaurant_id = cursor.lastrowid
        # 2. Create Restaurant Manager
        password_hash = bcrypt.hashpw(password.encode("utf-8"), bcrypt.
   gensalt())
        manager_query = """
            INSERT INTO Restaurant_Manager (name, surname, email,
   password, managesId)
            VALUES (%s, %s, %s, %s, %s)
        """
```

```
        manager_values = (manager_first_name, manager_last_name, email, password_hash, new_restaurant_id)
        cursor.execute(manager_query, manager_values)
        db.commit()
```

Listing 6: Restaurant Registration

5.2.2 Read - Restaurant and Restaurant Manager

This retrieves specific details about the logged-in manager and the restaurant they manage to populate the dashboard.

```
@restaurant.route('/dashboard')
def restaurant_dashboard():
    # ...
    try:
        query = """
            SELECT
                r.name as restaurant_name, r.city, r.address, r.cuisine, r.phone, r.description, r.photo_url,
                rm.name as manager_first_name, rm.surname as manager_last_name, rm.email
            FROM Restaurant r
            JOIN Restaurant_Manager rm ON r.r_id = rm.managesId
            WHERE r.r_id = %s AND rm.rm_id = %s
        """
        cursor.execute(query, (r_id, manager_id))
        data = cursor.fetchone()
```

Listing 7: Restaurant Search

5.2.3 Update - Restaurant and Restaurant Manager

This handles updates to both the Restaurant table (e.g., cuisine, address) and the Restaurant_Manager table (e.g., email, password) in a single request transaction.

```
@restaurant.route('/update', methods=['POST'])
def restaurant_update():
    # ...
    try:
        # Update Restaurant table
        r_query = """
            UPDATE Restaurant
            SET name=%s, city=%s, address=%s, cuisine=%s, phone=%s, description=%s
            WHERE r_id = %s
        """
        cursor.execute(r_query, (restaurant_name, city, address, cuisine, phone, description, r_id))
        # Update Restaurant_Manager table
        # ... (Logic to optionally include password update) ...
        rm_query = f"UPDATE Restaurant_Manager SET {', '.join(rm_query_parts)} WHERE rm_id = %s"
        cursor.execute(rm_query, tuple(rm_values))
        db.commit()
```

Listing 8: Restaurant Update

5.2.4 Delete - Restaurant and Restaurant Manager

Deletes a specific manager account. It includes a safety check to ensure the last manager of a restaurant cannot delete themselves, preventing a restaurant from being "orphaned."

```
@restaurant.route('/api/managers/delete', methods=['POST'])
def delete_manager():
    # ...
    try:
        # Check how many managers are left
        cursor.execute("SELECT COUNT(*) as count FROM Restaurant_Manager
    WHERE managesId = %s", (r_id,))
        # ...
        if manager_count <= 1:
            return jsonify({"error": "Cannot delete the last manager of
   the restaurant."}), 403

        cursor.execute("DELETE FROM Restaurant_Manager WHERE rm_id = %s"
   , (rm_id_to_delete,))
        db.commit()
```

Listing 9: Restaurant Manager Delete

Deletes the restaurant record. Note that depending on database cascading rules, this often deletes all associated managers, menus, and orders automatically.

```
@restaurant.route('/api/managers/delete', methods=['POST'])
@restaurant.route('/delete', methods=['POST'])
def restaurant_delete():
    # ...
    try:
        cursor.execute("DELETE FROM Restaurant WHERE r_id = %s", (r_id,)
   )
        if cursor.rowcount == 1:
            db.commit()
```

Listing 10: Restaurant Delete

5.3 Positions Module

5.3.1 Create - Positions

This operation allows a loggedin restaurant manager to post a new job opening. It accepts details like payment, experience required, and city. It defaults isOpen to TRUE.

```
@restaurant.route('/positions/create', methods=['POST'])
def create_position():
    # ... (Authentication check) ...
    # ... (Data validation) ...
    try:
        # ...
        # Insert new position
        cursor.execute("""
            INSERT INTO Positions (r_id, city, req_exp, req_rating,
   payment, isOpen)
            VALUES (%s, %s, %s, %s, %s, TRUE)
        """, (r_id, city, req_exp, req_rating, payment))

```

```
13          db.commit()
```

Listing 11: Positions Create

5.3.2 Read - Positions

This endpoint fetches all active job listings (isOpen = TRUE) specifically for the logged-in restaurant, ordered by the most recently created.

```
@restaurant.route('/api/positions')
def get_positions():
    # ... (Authentication check) ...
    try:
        cursor.execute("""
            SELECT p_id, city, payment, req_exp, req_rating, created_at
            FROM Positions
            WHERE r_id = %s AND isOpen = TRUE
            ORDER BY created_at DESC
        """, (r_id,))

        positions = cursor.fetchall()
```

Listing 12: Positions Read

5.3.3 Delete - Positions

This operation allows a manager to remove a job listing. Ideally, this performs a hard delete from the database effectively cancelling the position. It includes a security check to ensure the position belongs to the requester's restaurant.

```
@restaurant.route('/api/positions/delete', methods=['POST'])
def delete_position():
    # ... (Authentication check) ...
    try:
        # Verify the position belongs to the restaurant before deleting
        cursor.execute("SELECT r_id FROM Positions WHERE p_id = %s", (
    p_id_to_delete,))
        position = cursor.fetchone()

        # ... (Ownership validation) ...

        # Proceed with deletion
        cursor.execute("DELETE FROM Positions WHERE p_id = %s", (
    p_id_to_delete,))
        db.commit()
```

Listing 13: Positions Delete

5.4 Order Module (Yavuz Selim Yağlıca)

5.4.1 Create - Place Order with Automatic Courier Assignment

```
@order.route("/create", methods=["POST"])
def make_an_order():
    # Find available courier (least busy, highest rated)
    c_id = find_available_courier(cursor, r_id)
```

```
5
6       cursor.execute("""
7           INSERT INTO Orders
8           (user_id, r_id, m_id, c_id, sales_amount, sales_qty, currency,
    IsDelivered)
9           VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
10      """, (user_id, r_id, m_id, c_id, (qty * price), qty, "INR", 0))
11
12      o_id = cursor.lastrowid
13      t_id = create_task(cursor, o_id, c_id, user_id, m_id)  # Create
    delivery task
14      db.commit()
```

Listing 14: Order Creation with Task

5.4.2 Update - Trust-Weighted Rating System

```
1  # Trust-weighted formula prevents gaming the system
2  # new_rating = (current * (count + 3) + new) / (count + 4)
3  cursor.execute("""
4      UPDATE orders o
5      JOIN courier c ON c.c_id = o.c_id
6      JOIN restaurant r ON r.r_id = o.r_id
7      SET
8          c.rating = CASE
9              WHEN o.courier_rate IS NULL
10                 THEN (c.rating * (c.ratingCount + 3) + %s) / (c.
    ratingCount + 4)
11             ELSE (c.rating * (c.ratingCount + 3) - o.courier_rate + %s)
    / (c.ratingCount + 3)
12         END,
13         c.ratingCount = CASE
14             WHEN o.courier_rate IS NULL THEN c.ratingCount + 1
15             ELSE c.ratingCount
16         END,
17         o.courier_rate = %s
18     WHERE o.o_id = %s
19 """, (courier_rate, courier_rate, courier_rate, o_id))
```

Listing 15: Complex Rating Update Query

5.5 Menu & Food Module (Hasan)

5.5.1 Create - Menu with Data Normalization Integrity

The creation process strictly follows 3NF principles by separating logical food items from their restaurant-specific menu implementations. Before creating a menu entry, the system checks the global `Food` table. If the item (e.g., "Burger") exists, we reuse its `f_id`; otherwise, a new item is created. This "Smart Reuse Strategy" prevents redundant string storage and ensures database cleaner normalization.

```
1  @menu.route("/", methods=["POST"])
2  def create_menu_item():
3      data = request.get_json()
4      # Extract vars
5      f_id = data.get("f_id")
```

```
    food_nm = data.get("food_name")
    veg = data.get("veg_or_non_veg")

    # 1. Resolve Food ID (Reuse logic)
    if not f_id:
        f_id = _ensure_food(cur, food_nm, veg)

    # 2. Link Restaurant to Food in Menu Table
    cur.execute(
        """
        INSERT INTO Menu (menu_id, r_id, f_id, cuisine, price)
        VALUES (%s, %s, %s, %s, %s)
        """,
        (menu_id, r_id, f_id, cuisine, price),
    )
    return jsonify({"message": "Menu item created", "m_id": menu_id}),
   201
```

Listing 16: Menu Item Creation Endpoint

Helper Function for Normalization: The `_ensure_food` helper referenced above handles the deduplication logic:

```
def _ensure_food(cur, item, veg):
    """Reuse existing food or create new entry"""
    # 1. Check Global Catalog (Dedup) reusing helper
    fid = _find_food_by_name(cur, item)
    if fid:
        return fid

    # 2. Create New if not found
    fid = _next_food_id(cur)  # Generates 'fd1', 'fd2'...
    cur.execute(
        "INSERT INTO Food (f_id, item, veg_or_non_veg) VALUES (%s,%s,%s)
    ",
        (fid, item, veg or "Non-Veg")
    )
    return fid
```

Listing 17: Helper: Ensure Food Exists or Create

5.5.2 Read - Global Food Search with Query Optimization

The food search uses `GROUP BY` aggregation to improve the search space and user experience. Since different restaurants may map multiple `f_id`s to similar names, raw SELECTs would return duplicates. By grouping by semantic uniqueness (`item` + `veg_or_non_veg`), we reduce the payload size transferred to the client and provide a clean, standardized list for the autocomplete UI.

```
@food.route("/", methods=["GET"])
def search_foods():
    # Dynamic SQL construction
    sql = """
        SELECT item, veg_or_non_veg, MAX(f_id) as sample_id
        FROM Food
        WHERE 1=1
    """
    if q:
```

```
        sql += " AND item LIKE %s"

    # GROUP BY ensures the user sees 'Burger' only once (Clean UI)
    sql += " GROUP BY item, veg_or_non_veg ORDER BY item ASC LIMIT %s"

    cursor.execute(sql, params)
    return jsonify(cursor.fetchall())
```

Listing 18: Global Food Search Implementation

5.5.3 Update - Partial Updates and SQL Injection Security

We implemented a secure, dynamic query construction pattern for updates. Instead of string concatenation which is vulnerable to SQL Injection, we use parametric binding. The system supports "PATCH"-style partial updates, where only the modified fields (Price, Cuisine) are included in the `UPDATE` clause, improving efficiency by not rewriting unchanged data.

```
fields = []
if 'price' in data: fields.append("price=%s")
if 'cuisine' in data: fields.append("cuisine=%s")
# Explicitly constructing safe SQL string
sql = f"UPDATE Menu SET {', '.join(fields)} WHERE m_id=%s"
cur.execute(sql, (*params, m_id))
```

Listing 19: Dynamic Update Query

5.5.4 Delete - Referential Integrity and Cascading

Deletion is handled entirely by the database engine's referential integrity constraints, ensuring zero orphaned records.

1. **Menu Deletion:** Removes the row from 'Menu' table. The global 'Food' item remains untouched (safe for other restaurants).

2. **Restaurant Deletion:** We utilize `ON DELETE CASCADE` to automatically propagate deletions.

```
CREATE TABLE Menu (
    m_id INT PRIMARY KEY AUTO_INCREMENT,
    r_id INT,
    f_id VARCHAR(50), -- String ID (e.g., 'fd1')
    -- When Restaurant is deleted, auto-delete its Menu
    FOREIGN KEY (r_id) REFERENCES Restaurant(r_id) ON DELETE CASCADE,
    FOREIGN KEY (f_id) REFERENCES Food(f_id) ON DELETE CASCADE
);
```

Listing 20: SQL Schema Proof of Cascading Deletion

5.5.5 Backend Scalability (Future Proofing)

We implemented several backend endpoints that are fully functional but not yet exposed in the primary UI, ensuring the system is ready for future scaling: We deliberately implemented certain backend capabilities to anticipate future scaling requirements, ensuring the system architecture is robust beyond the current UI. The backend already includes

a fully functional `/menu/search` endpoint constructed with dynamic `WHERE 1=1` filtering logic. This allows for complex, headless queries—such as filtering by cuisine type or price range—ready for integration with a potential mobile application. Additionally, an administrative `create_food` endpoint permits the seeding of the global food catalog independent of restaurant-specific menus, establishing a solid foundation for standardized data management.

5.6 Courier & Task Module (Furkan)

5.6.1 Create - Task Helper Function

```
def create_task(cursor, o_id, c_id, user_id, m_id):
    """Creates delivery task in same transaction as Order"""
    cursor.execute("SELECT address FROM User WHERE user_id = %s", (user_id,))
    user_row = cursor.fetchone()

    cursor.execute("""
        INSERT INTO Task (o_id, c_id, user_id, m_id, user_address, status)
        VALUES (%s, %s, %s, %s, %s, %s)
    """, (o_id, c_id, user_id, m_id, user_row["address"], 0))
    task_id = cursor.lastrowid

    # Increment active task count
    cursor.execute("UPDATE Courier SET taskCount = taskCount + 1 WHERE c_id = %s", (c_id,))
    return task_id
```

Listing 21: Task Creation Within Transaction

5.6.2 Update - Complete Delivery (Multi-Table Update)

```
@courier.route('/tasks/complete/<int:task_id>', methods=['POST'])
def complete_task(task_id):
    # 1. Mark task completed
    cursor.execute("UPDATE Task SET status = 1 WHERE t_id = %s", (task_id,))

    # 2. Update order delivery status
    cursor.execute("UPDATE Orders SET IsDelivered = TRUE WHERE o_id = %s", (o_id,))

    # 3. Increment position deliveries
    cursor.execute("""
        UPDATE Positions SET deliveries_made = deliveries_made + 1
        WHERE c_id = %s AND r_id = %s
    """, (courier_id, r_id))

    # 4. Update courier statistics
    cursor.execute("""
        UPDATE Courier
        SET TotalDeliveries = TotalDeliveries + 1,
            taskCount = GREATEST(taskCount - 1, 0)
        WHERE c_id = %s
```

```
    """, (courier_id,))
    db.commit()
```

Listing 22: Delivery Completion with Statistics Update

6 Complex Queries

6.1 4-Table JOIN Query - User's Order History

This query retrieves complete history for the user's orders by joining 4 tables to show the necessary information.

```
SELECT O.*, R.name,C.name, C.surname,M.cuisine
            FROM Orders O
            JOIN Restaurant R ON R.r_id = O.r_id
            JOIN Courier C ON C.c_id = O.c_id
            JOIN Menu M ON M.m_id = O.m_id
            WHERE O.user_id = %s ORDER BY O.order_date DESC
```

Listing 23: 4-Table JOIN Query

6.2 5-Table JOIN Query - Delivery History

This query retrieves the complete delivery history using 3 INNER JOINs and 2 LEFT OUTER JOINs to preserve data integrity when menu items are deleted.

```
SELECT
    t.t_id, t.task_date, t.user_address AS delivery_address, t.status,
    o.o_id, o.sales_qty, o.sales_amount, o.courier_rate, o.menu_rate,
    u.user_id, u.name AS customer_name, u.city AS customer_city,
    r.r_id, r.name AS restaurant_name, r.cuisine AS restaurant_cuisine,
    m.m_id, m.price AS menu_price,
    f.f_id, f.item AS food_name, f.veg_or_non_veg
FROM Task t
INNER JOIN Orders o ON t.o_id = o.o_id
INNER JOIN User u ON t.user_id = u.user_id
INNER JOIN Restaurant r ON o.r_id = r.r_id
LEFT OUTER JOIN Menu m ON t.m_id = m.m_id
LEFT OUTER JOIN Food f ON m.f_id = f.f_id
WHERE t.c_id = %s
ORDER BY t.task_date DESC
```

Listing 24: 5-Table JOIN Query

6.3 GROUP BY - Restaurant Statistics

```
SELECT
    r.r_id, r.name AS restaurant_name, r.cuisine,
    COUNT(t.t_id) AS delivery_count,
    SUM(o.sales_amount) AS total_earnings,
    AVG(o.courier_rate) AS avg_rating
FROM Task t
INNER JOIN Orders o ON t.o_id = o.o_id
INNER JOIN Restaurant r ON o.r_id = r.r_id
WHERE t.c_id = %s AND t.status = 1
```

```
GROUP BY r.r_id, r.name, r.cuisine
ORDER BY delivery_count DESC
```

Listing 25: Aggregated Statistics per Restaurant

6.4 Nested Subquery - Restaurant Leaderboard

This query implements position-based performance tracking using the `hired_at` column to only count deliveries made after a courier joined their current position.

```
SELECT
    c.c_id, c.name AS courier_name, c.surname AS courier_surname,
    c.rating AS courier_rating,
    COUNT(t.t_id) AS total_deliveries,
    AVG(o.courier_rate) AS avg_delivery_rating,
    (COUNT(t.t_id) * COALESCE(AVG(o.courier_rate), 0)) AS score
FROM Courier c
INNER JOIN Positions p ON c.c_id = p.c_id AND c.r_id = p.r_id
INNER JOIN Task t ON c.c_id = t.c_id
INNER JOIN Orders o ON t.o_id = o.o_id
WHERE t.status = 1
  AND t.task_date >= p.hired_at  -- Only count post-hire deliveries
  AND c.r_id = (
      SELECT r_id FROM Courier WHERE c_id = %s  -- Nested subquery
  )
GROUP BY c.c_id, c.name, c.surname, c.rating
ORDER BY score DESC
LIMIT 10
```

Listing 26: Leaderboard with Nested Subquery

6.5 Bestseller & Top Courier Spotlight

This query is designed to identify the "Top Courier for the Best-Selling Item" at a specific restaurant. It answers the question: "Who is the single courier that has delivered our most popular dish the most times?"

```
SELECT
    c.name AS courier_name, f.item AS food_item,
    COUNT(o.o_id) AS delivery_count,
    AVG(o.courier_rate) AS avg_rating_for_this_item
FROM Orders o
JOIN Courier c ON o.c_id = c.c_id
JOIN Menu m ON o.m_id = m.m_id
JOIN Food f ON m.f_id = f.f_id
WHERE o.r_id = %s
  AND m.f_id = (
      SELECT m2.f_id FROM Orders o2
      JOIN Menu m2 ON o2.m_id = m2.m_id
      WHERE o2.r_id = %s
      GROUP BY m2.f_id
      ORDER BY COUNT(o2.o_id) DESC
      LIMIT 1
  )
GROUP BY c.c_id, f.item
ORDER BY delivery_count DESC, avg_rating_for_this_item DESC
```

```
LIMIT 1
```

Listing 27: Complex Nested Subquery for Bestseller Analysis

The query first runs a subquery to determine the food item with the highest order count for that specific restaurant. It then filters the main order history to isolate transactions for this popular item, joins them with courier details, and aggregates the data to calculate delivery counts and average ratings per courier. Finally, it sorts the results to return the single courier with the highest delivery volume for that specific item, effectively spotlighting the primary driver for the restaurant's most successful product.

6.6 Market Gap Analysis ("Missed Opportunities")

This advanced analytical query serves as a Business Intelligence tool for restaurant managers. It identifies high-demand items in the specific city that the restaurant does not currently offer. The query employs `Set Theory` logic via `NOT IN` to calculate the "Difference" between the city's popular set and the restaurant's current set, ordered by volume (‘COUNT‘).

```
SELECT
    MAX(f.f_id) as f_id,
    f.item,
    f.veg_or_non_veg,
    COUNT(o.o_id) as demand_count
FROM Orders o
JOIN Menu m ON o.m_id = m.m_id
JOIN Food f ON m.f_id = f.f_id
JOIN Restaurant r ON o.r_id = r.r_id
WHERE r.city = %s           -- 1. Scope: My City
  AND r.r_id != %s          -- 2. Exclude: My Own Sales
  AND f.item NOT IN (       -- 3. Filter: Set Difference
      SELECT f2.item
      FROM Menu m2
      JOIN Food f2 ON m2.f_id = f2.f_id
      WHERE m2.r_id = %s
  )
GROUP BY f.item, f.veg_or_non_veg
ORDER BY demand_count DESC
LIMIT 5
```

Listing 28: Market Gap Analysis Query

6.7 Window Function - Top Cuisine Per City

```
SELECT city, cuisine, total_sold
FROM (
    SELECT
        r.city, m.cuisine,
        SUM(o.sales_qty) AS total_sold,
        RANK() OVER (PARTITION BY r.city ORDER BY SUM(o.sales_qty) DESC)
    AS rnk
    FROM Orders o
    JOIN Restaurant r ON o.r_id = r.r_id
    JOIN Menu m ON o.m_id = m.m_id
    WHERE o.IsDelivered = TRUE
```

```
    GROUP BY r.city, m.cuisine
) ranked
WHERE rnk = 1
```

Listing 29: RANK Window Function

7 User Interface Features

7.1 Main Page

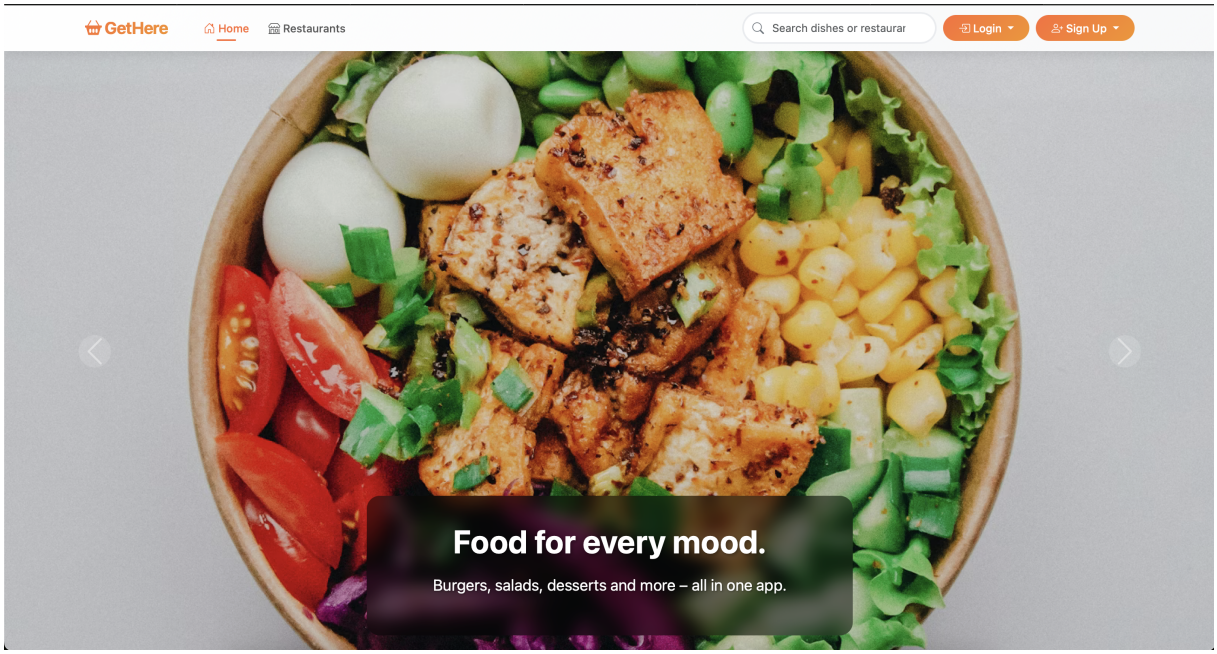


Figure 2: Welcome Page of GetHere

This is the main landing page for GetHere, designed to engage users immediately with a full-screen, visually rich "Hero Carousel" showcasing delicious food imagery and value propositions. It serves as the primary gateway for all user types, featuring a responsive navigation bar with dedicated login/signup dropdowns for users, couriers, and restaurants. The page also integrates a "Platform Statistics" section (dynamically loaded via JavaScript) to build trust by displaying live metrics like total orders, top cuisines, and popular restaurants, creating an active and vibrant marketplace atmosphere.

7.2 User Module Interface

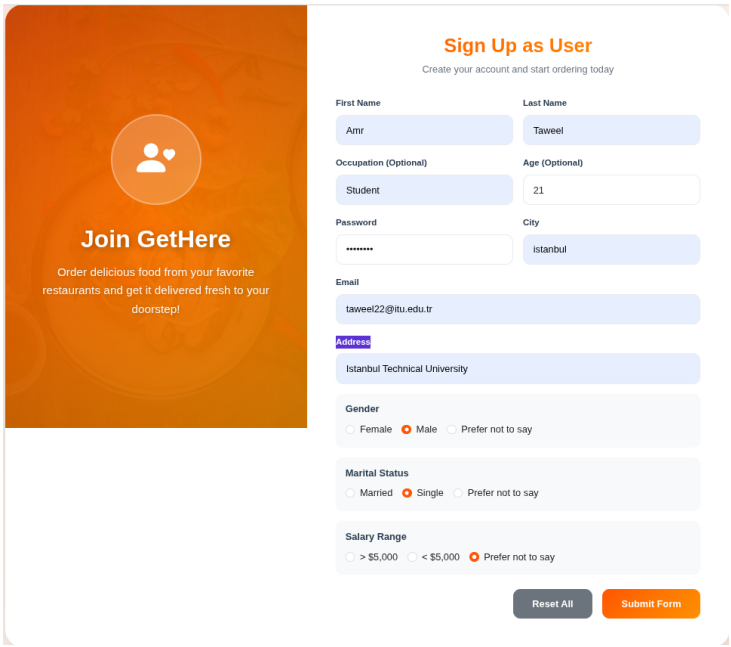


Figure 3: User Registration Form - Collects personal information including name, occupation, address, gender, marital status, and salary range

The user sign-up page serves as a secure identity initialization module, responsible for registering new users and provisioning their authentication credentials within the system. It gathers core user attributes such as name, email, and encrypted password, enforcing uniqueness and data integrity through client-side HTML5 validation and backend verification mechanisms.

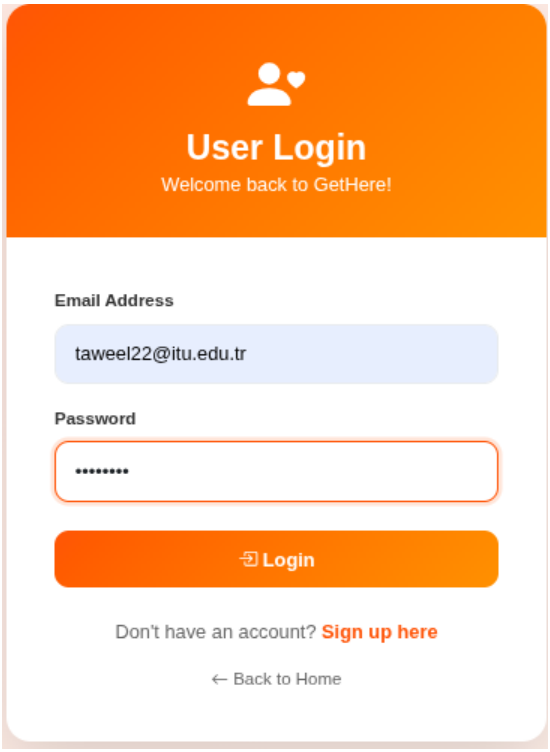


Figure 4: User Login Page - Email and password authentication with bcrypt verification

The user login page operates as a secure and centralized access point for end users, providing authenticated entry to the platform while maintaining strict separation from restaurant and administrative accounts. It enforces robust authentication mechanisms by validating user credentials against securely hashed passwords and establishing a personalized session context that governs user-level permissions and interactions. To improve usability and reduce access friction, the interface integrates immediate input validation to prevent empty or malformed submissions and offers clear navigation options, enabling new users to seamlessly access the registration page or redirect back to the main application if the login page was reached unintentionally.

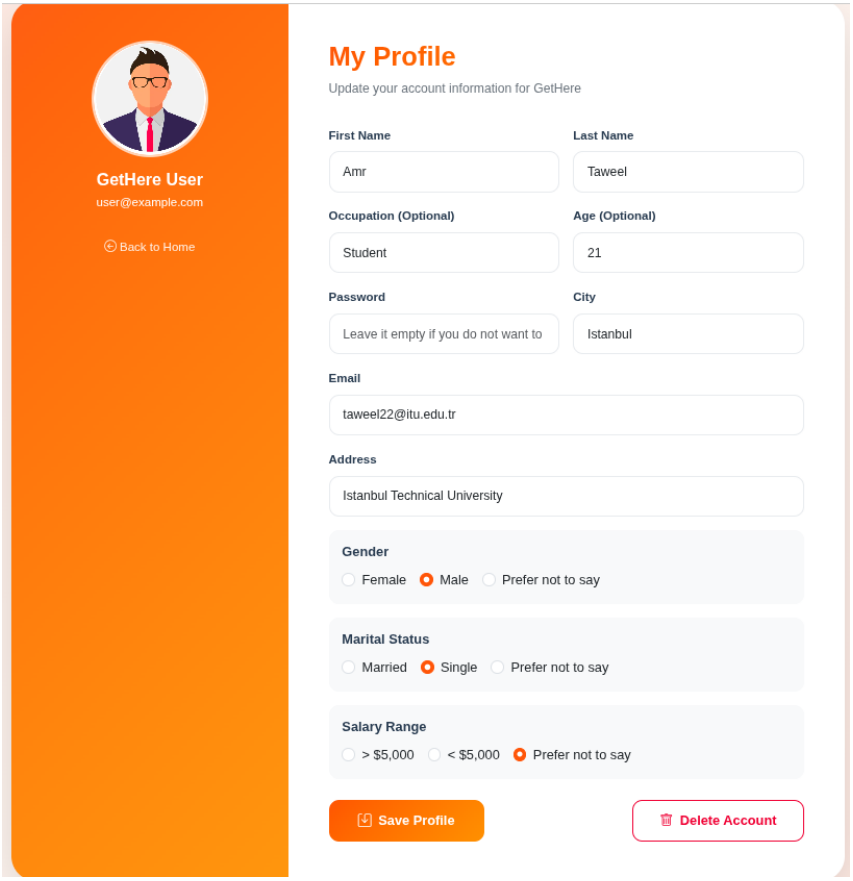


Figure 5: User Profile Management - Users can update their information or delete their account

The user profile page serves as a centralized account management interface, enabling authenticated users to securely view, update, and maintain their personal information within the system. It allows users to modify both mandatory attributes (such as name, email, and city) and optional demographic details (including occupation, age, gender, marital status, and salary range), providing flexibility while preserving data completeness. To enhance security, password updates are handled conditionally—only applied when explicitly provided—preventing unintended credential changes. Client-side validation ensures data integrity by restricting invalid or empty submissions before updates are committed to the database.In addition to profile updates, the page incorporates a dedicated account deletion mechanism, allowing users to permanently remove their account through an explicit and clearly labeled action.

7.3 Restaurant Module Interface

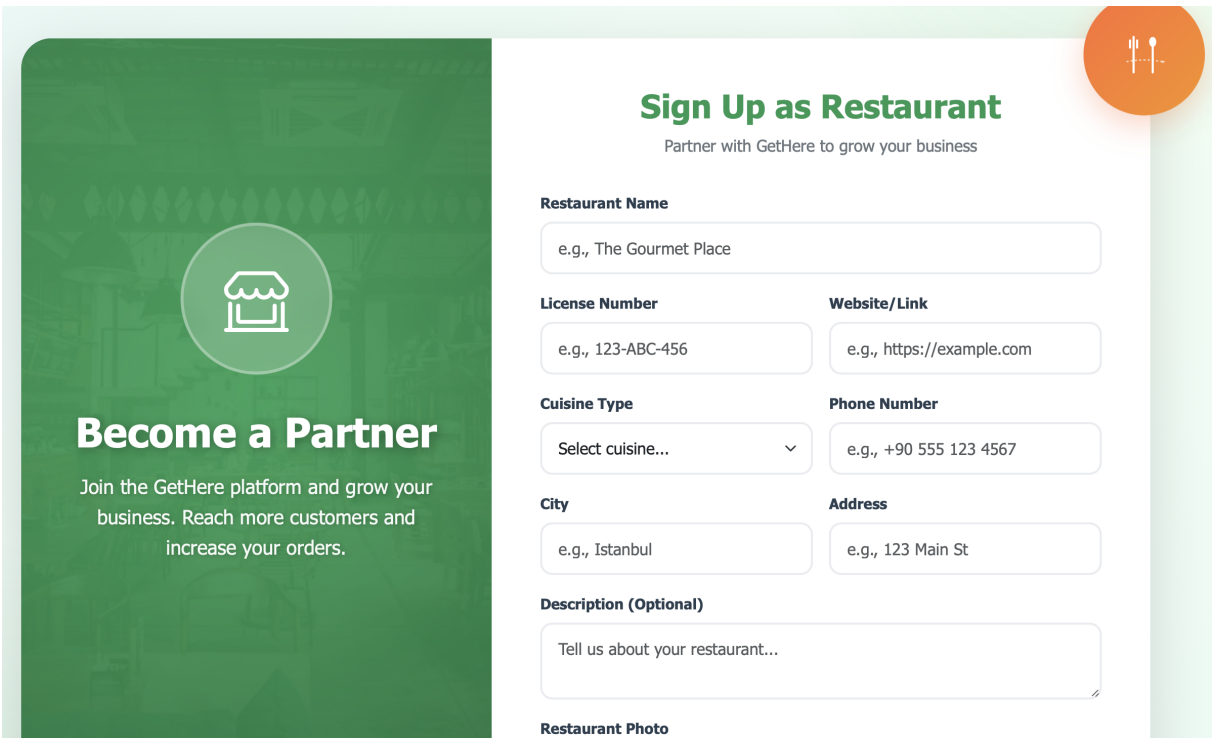


Figure 6: Restaurant Sign Up Page

The restaurant sign-up page serves as a unified dual-entity creation portal, allowing a business to simultaneously establish its public profile and its administrative credential in a single workflow. It captures comprehensive "Restaurant" data—including cuisine type, license number, physical address, and a promotional photo—while parallelly collecting "Manager" credentials (name, email, secure password) to ensure the new business has immediate oversight capabilities. The interface is built with a responsive split-layout design that combines a visually appealing marketing banner with a structured, multi-column form, utilizing HTML5 validation and file upload support to ensure rich, accurate data entry right from the start.

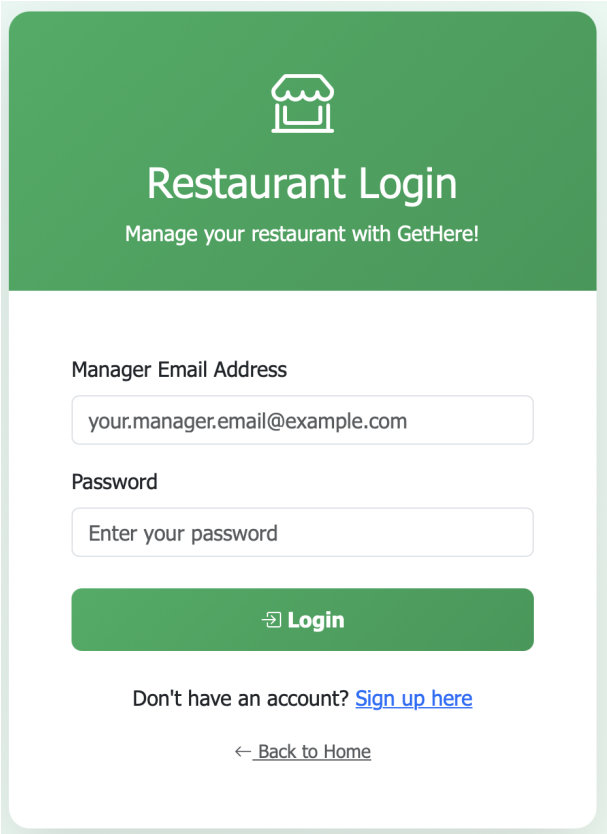


Figure 7: Restaurant Login Page

The restaurant login page functions as a dedicated, secure gateway for business administration, strictly isolating manager access from standard customer accounts to protect sensitive operational data. It ensures security through robust authentication, verifying credentials against hashed passwords and establishing a specific session context that grants the manager distinct privileges tied strictly to their own restaurant. To enhance the user experience, the system incorporates immediate input validation to prevent empty submissions and provides clear navigation pathways, allowing new partners to easily find the registration form or users to return to the main application if they arrived there by mistake.

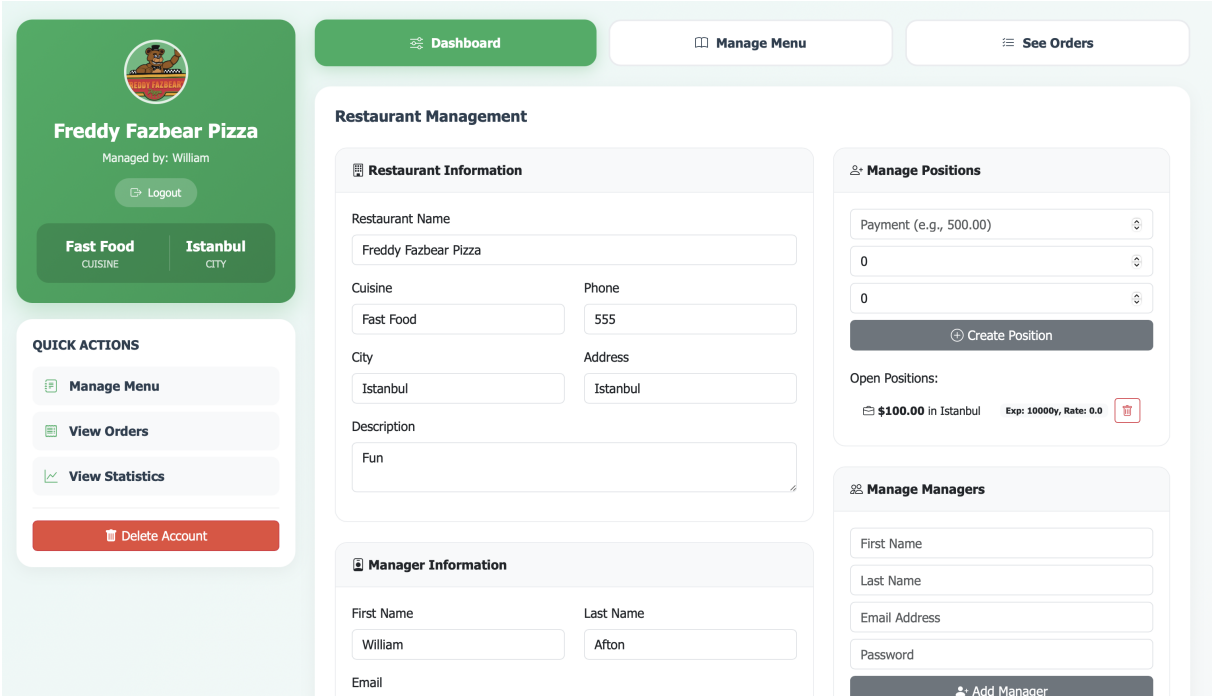


Figure 8: Restaurant Dashboard

The restaurant dashboard provides a comprehensive management interface:

- **Restaurant Information:** Update details such as name, cuisine type, contact phone, physical address, and description.
- **Position Management:** Create and manage job listings, specifying payment amounts, required years of experience, and minimum rating criteria.
- **Manager Administration:** Add new administrative staff credentials or remove existing managers from the system.
- **Quick Actions:** Access direct links to critical operational areas like menu editing, order tracking, and performance statistics.
- **Account Security:** Manage personal manager profile details (name, email, password) and access critical account deletion controls.

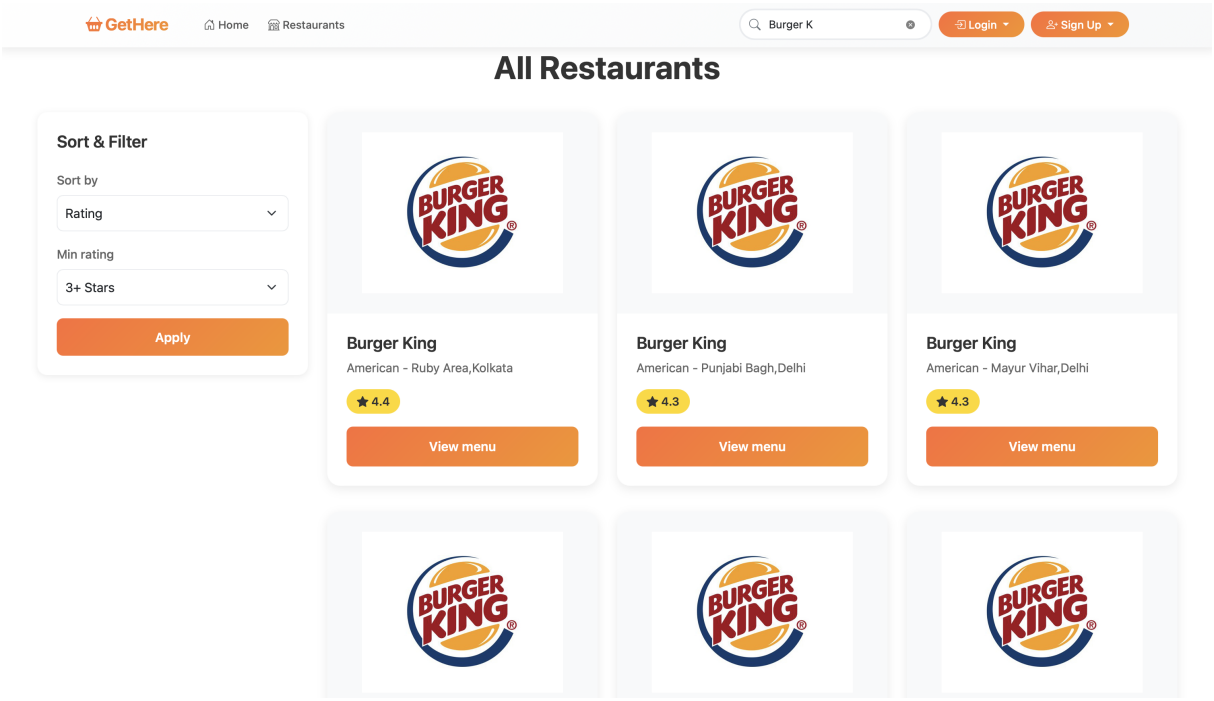


Figure 9: Restaurant Search with Filtering

The home page features a dynamic, real-time search system that allows users to instantly discover restaurants without reloading the page. As users type in the search bar or adjust filters (like rating or popularity), a JavaScript function captures these inputs and queries the backend API. The page then automatically updates the results grid, seamlessly transitioning from the hero banner to a custom list of matching restaurant cards that display key details, ratings, and menu links.

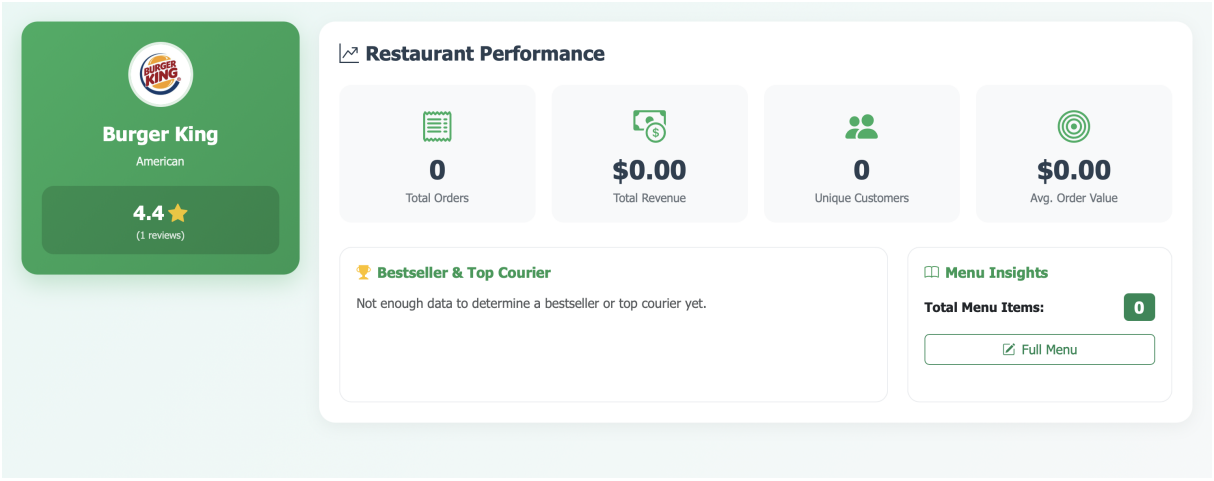


Figure 10: Restaurant Information Page

The Restaurant info page acts as a detailed performance dashboard for a specific restaurant. It provides analytic insights by displaying key metrics (KPIs) such as total orders, revenue, unique customer count, and average order value in a clean grid layout. Additionally, it highlights specific success stories, such as identifying the "Top Courier" for the best-selling menu item, while also offering a quick overview of menu size and direct links to full menu management.

7.4 Menu & Food Module Interface

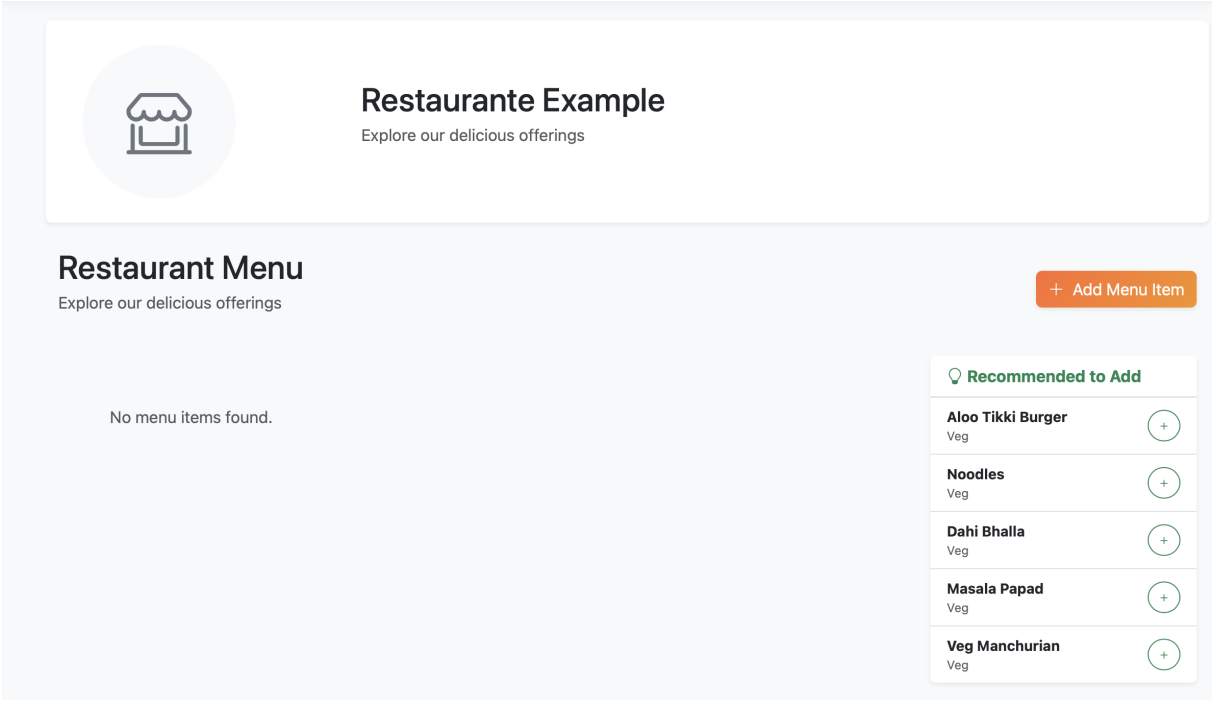
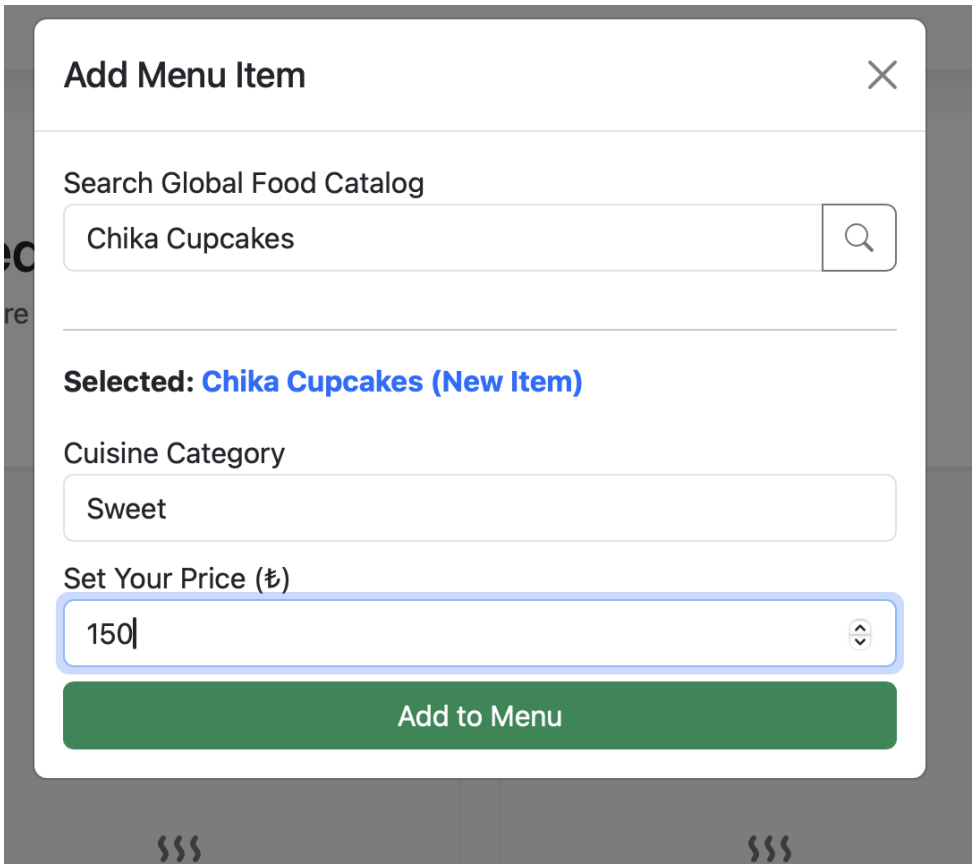
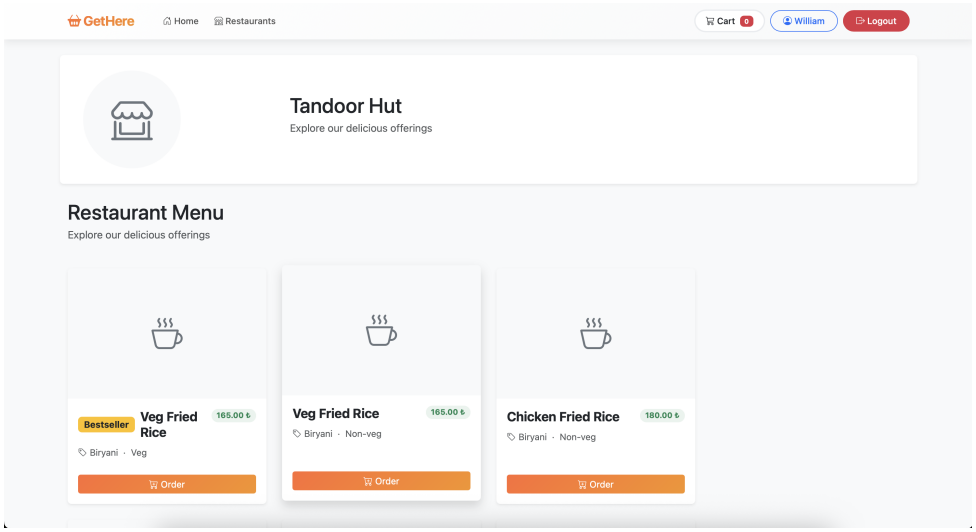


Figure 11: Market Gap Analysis ("Missed Opportunities") showing high-demand items not offered by the restaurant.



(a) Add Menu Modal (Smart Search)



(b) Menu List with Bestseller Tags

Figure 12: Menu Management Interface: Adding items (top) and viewing bestsellers (bottom)

Key features of the Menu system designed for efficiency and user experience: To maximize efficiency, we designed the Menu system around a **Modal-Based Workflow** that keeps the manager within the context of their dashboard. A key component of this experience is the **Asynchronous Global Search**, a "Smart Search" feature powered by AJAX. As the user types, the system queries the global catalog in real-time, encouraging the reuse

of valid `f_id`s preventing duplication. Visual feedback is immediate; adding a new menu entry or deleting an item triggers instant updates without a page reload, while data-driven "Missed Opportunities" cards provide actionable business intelligence directly in the UI. Additionally, a "Bestseller" badge is automatically calculated and displayed for the highest-performing menu item based on live order history.

7.5 Order Module Interface

The order module connects users, couriers and restaurants on the platform. Providing:

- Order creation for users
- Order viewing page for restaurants
- Task creation for couriers
- Updating the orders for users
- Rating of the restaurants and couriers via users
- Additional platform order statistics

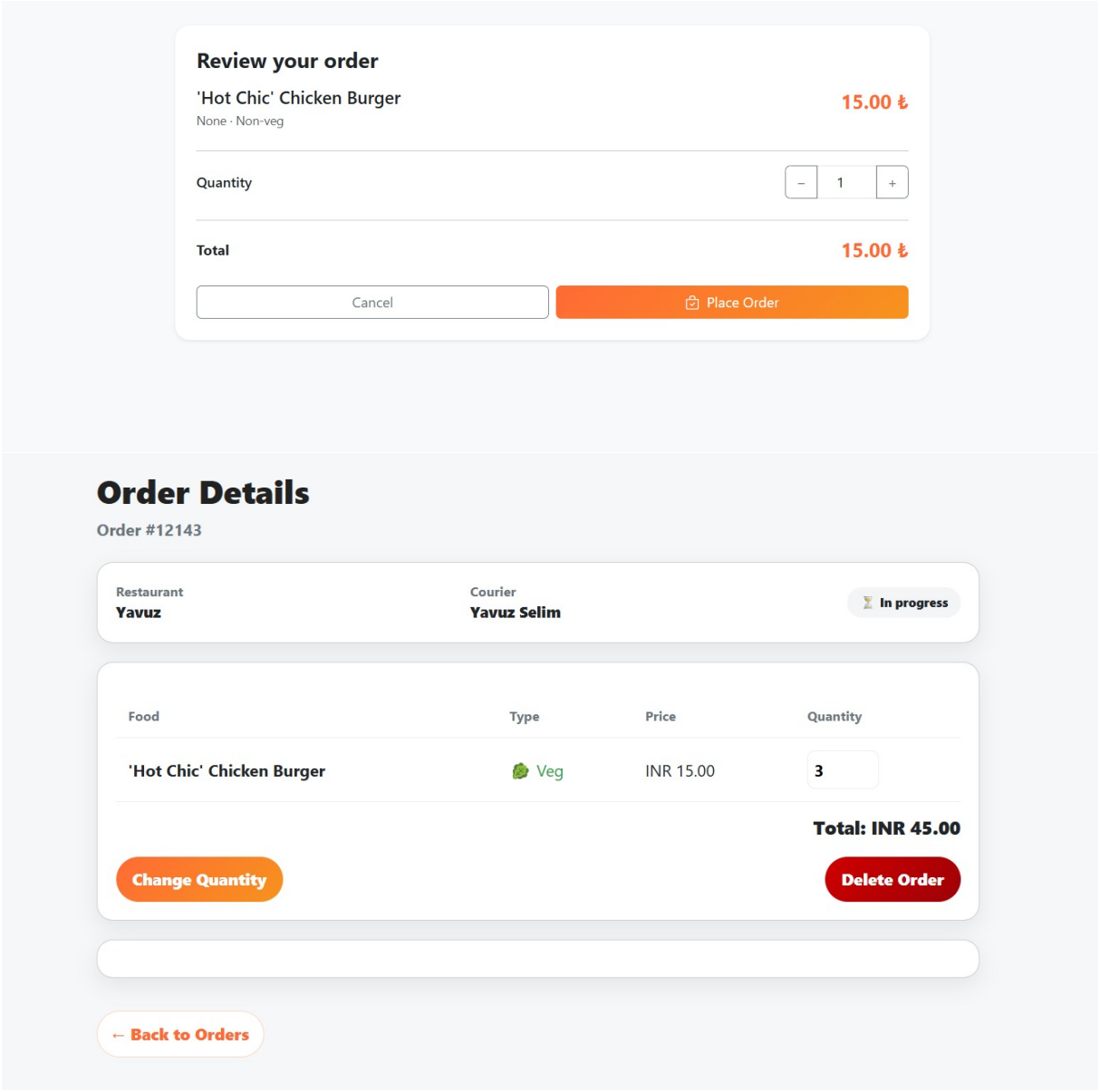


Figure 13: Order Creation (top), Order Update/Delete based on delivery status (bottom)

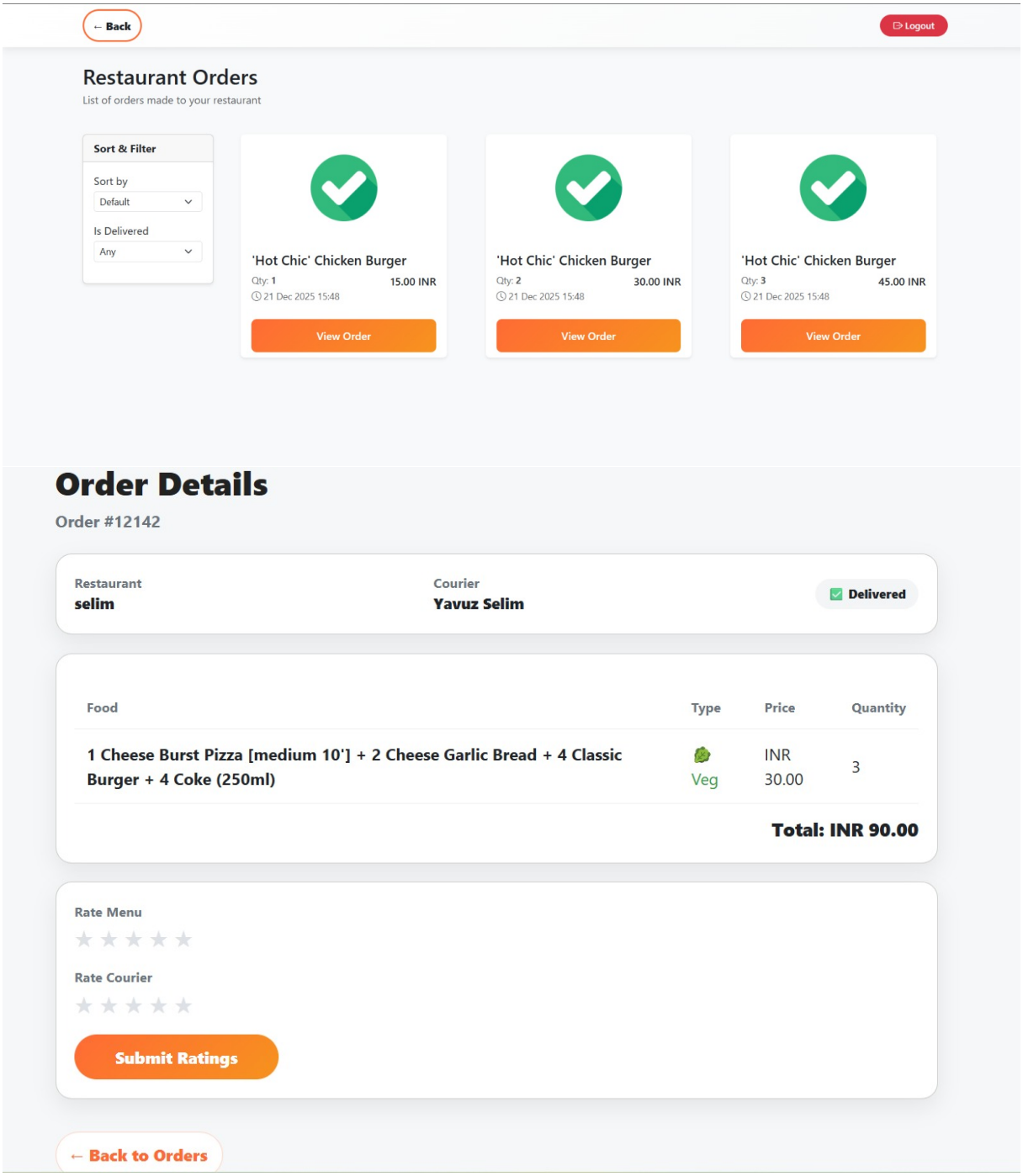


Figure 14: Restaurant Orders View with Filtering and Sorting (top) and Order Rating Updating based on delivery status (bottom)

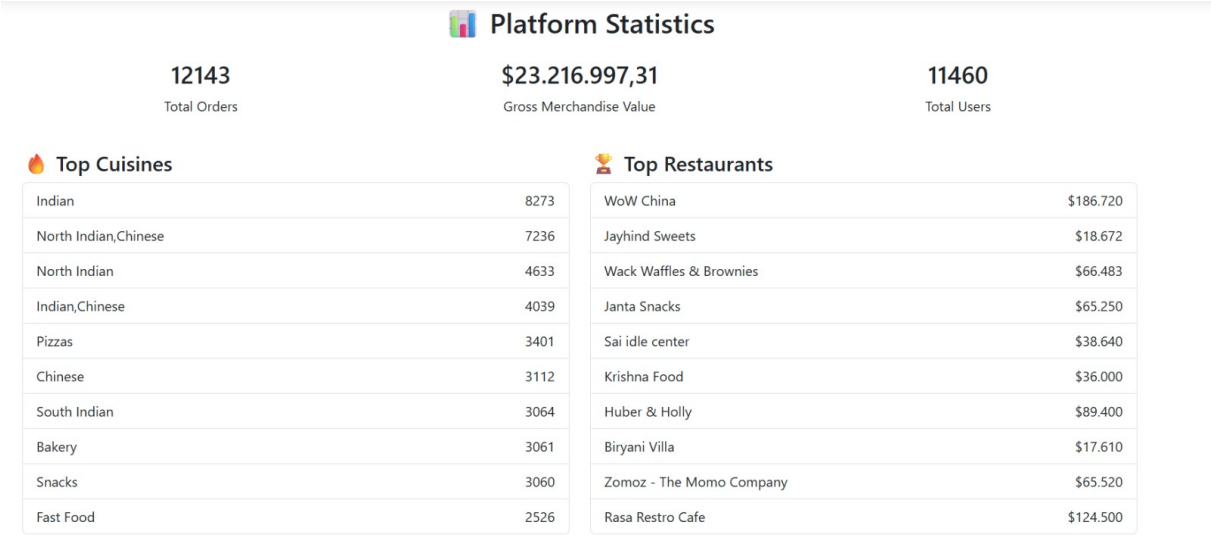


Figure 15: Platform-Wide Statistics Dashboard showing 12,143 orders, $23M GMV, 11,460 users, Top Cuisines, and Top Restaurants

7.6 Courier Module Interface

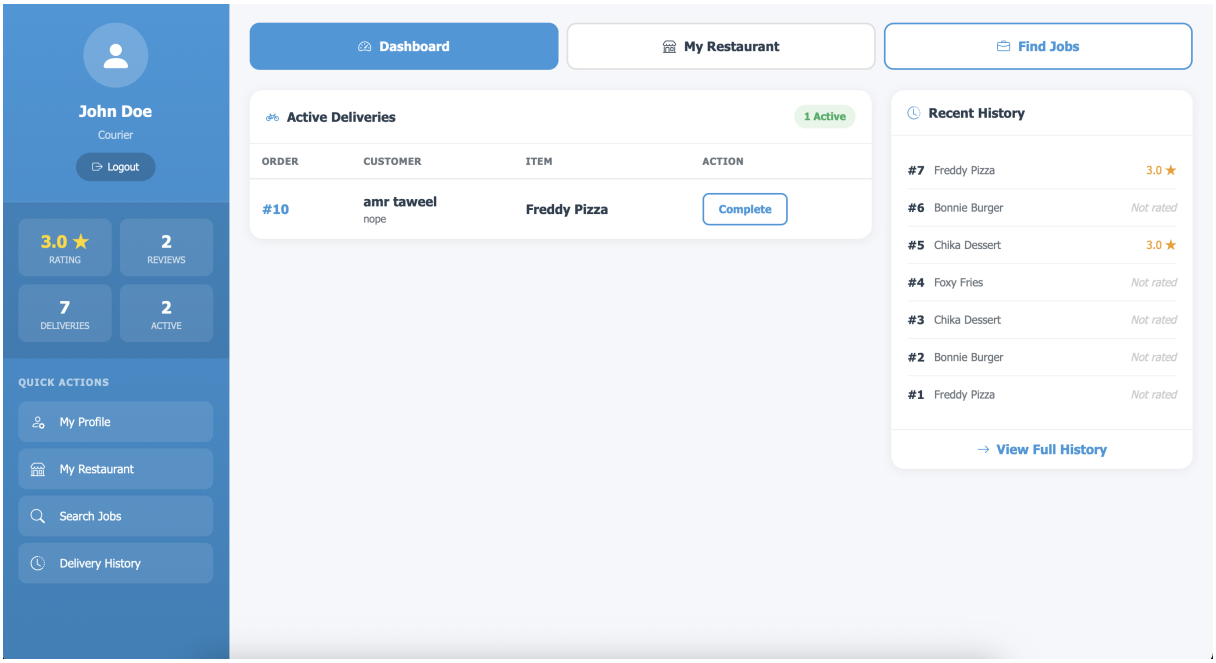


Figure 16: Courier Dashboard showing statistics and active deliveries (top), Dynamic Courier Selection Query (middle), and Job Search with Top Paying Positions (bottom)

The courier dashboard displays:

- **Profile Statistics:** Rating, reviews count, total deliveries, active tasks
- **Active Deliveries:** List of ongoing tasks with customer info and completion button
- **Quick Actions:** Navigation to profile, restaurant, job search, and history

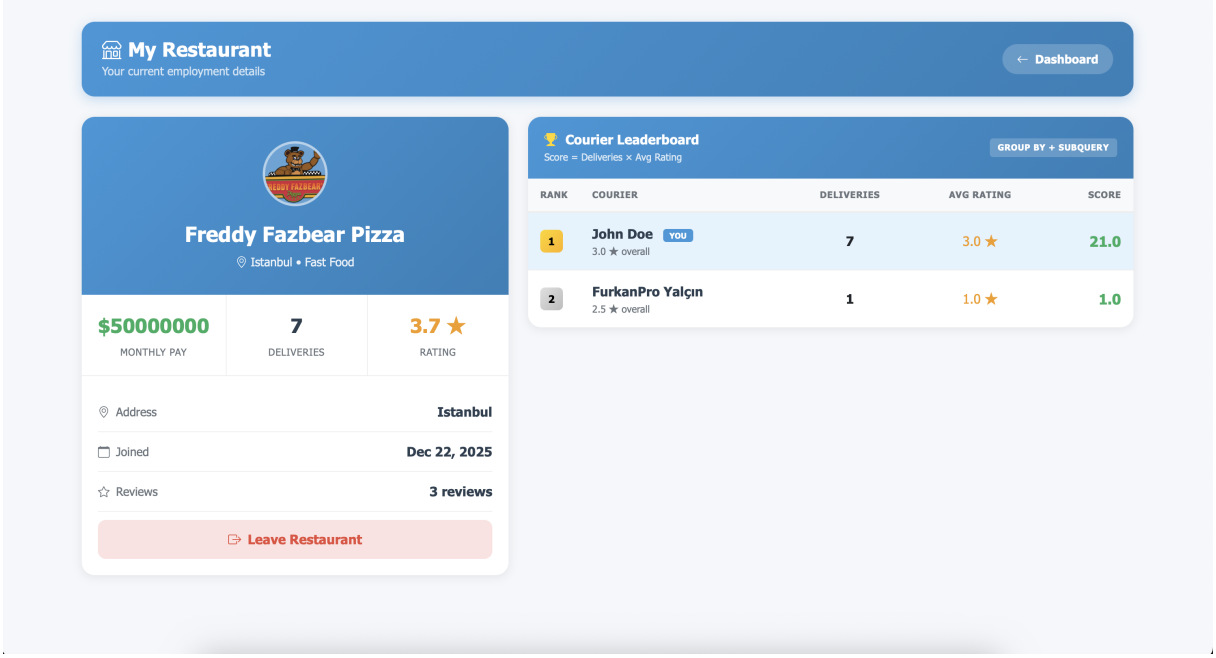


Figure 17: My Restaurant Page with Leaderboard Query (top) and Leave Restaurant Validation preventing departure with pending tasks (bottom)

Key courier features:

- **Leaderboard:** Score = total_deliveries × avg_rating, filtered by `hired_at`
- **Leave Validation:** Cannot leave with unfinished deliveries (shows pending count)
- **Position Details:** Monthly pay, deliveries made, join date, reviews count

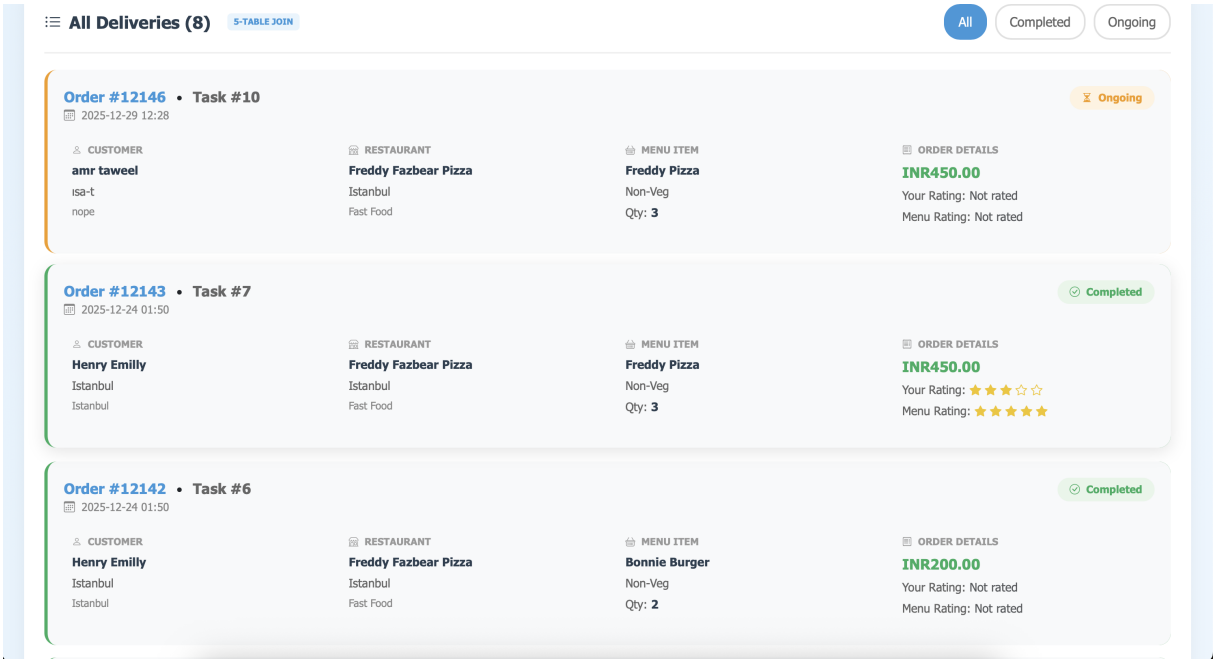


Figure 18: Delivery History Page with 5-Table JOIN and GROUP BY statistics (top) and Courier Profile Page with editable fields (bottom)

The Delivery History page demonstrates:

- **Summary Cards:** Completed count, ongoing count, total earnings, average rating
- **Deliveries by Restaurant:** GROUP BY aggregation showing per-restaurant statistics
- **All Deliveries:** 5-Table JOIN showing customer, restaurant, menu, food, and order details
- **Filter Tabs:** All, Completed, Ongoing deliveries

8 Challenges and Solutions

8.1 Challenge 1: Position-Based Performance Tracking

Problem: When a courier leaves and rejoins a restaurant, the leaderboard showed all-time deliveries instead of current position performance.

Root Cause: Query used `Position.created_at` (job listing creation date) rather than actual hire date.

Solution: Added `hired_at` column:

```sql
ALTER TABLE Positions ADD COLUMN hired_at TIMESTAMP NULL DEFAULT NULL;

-- Set during position application:
UPDATE Positions
SET c_id = %s, isOpen = FALSE, hired_at = CURRENT_TIMESTAMP
WHERE p_id = %s;

-- Leaderboard filter:
WHERE t.task_date >= p.hired_at
```

8.2 Challenge 2: Trust-Weighted Rating System

Problem: New couriers with 0 rating couldn't get jobs.

Solution: Implemented trust-weighted formula where new couriers start with 3.0 rating:

$$new\_rating = \frac{current \times (count + 3) + given}{count + 4} \tag{1}$$

8.3 Challenge 3: Orphaned Tasks Prevention

Problem: Couriers could leave restaurants with unfinished deliveries.

Solution: Added validation check before allowing departure:

```python
cursor.execute("""
    SELECT COUNT(*) as pending_count FROM Task
    WHERE c_id = %s AND status = 0
""", (courier_id,))
if pending['pending_count'] > 0:
    return jsonify({"error": "Complete pending deliveries first"}), 400
```

8.4 Challenge 4: Data Consistency in Menu Deletion

Problem: Deleting menu items could break order history display.

Solution: Used LEFT OUTER JOINs in history queries and ON DELETE SET NULL constraint to preserve order records while allowing menu modifications.

8.5 Challenge 5: Lack of Data for Restaurant Manager

The *insert_ data.py* script addresses the lack of native restaurant manager data in the original Zomato dataset by programmatically generating synthetic admin accounts during the import process. After inserting each restaurant record, the script immediately creates a corresponding manager entry. It generates a placeholder email based on the restaurant's unique ID (e.g., 123@gmail.com), uses a hashed default password (123), and links this new manager directly to the restaurant via the managesId foreign key. This logic ensures that every restaurant imported from the dataset comes pre-equipped with a functional administrative account, enabling immediate testing and management of the imported entities.

8.6 Challenge 6: Normalization vs. Search Functionality

A significant architectural challenge arose from our strict adherence to 3NF, specifically the separation of the `Food`, `Menu`, and `Restaurant` tables. While conceptually clean, this separation complicated the user's search experience; a user searching for "Burger" expects to find restaurants serving burgers, yet the `Restaurant` table contains no such text. To solve this without compromising normalization, we implemented a "Universal Search" backend mechanism. This solution uses a single, optimized 3-way `LEFT JOIN` across `Restaurant`, `Menu`, and `Food` tables (`list_restaurants` endpoint). This unified view allows a single search term to simultaneously match against restaurant names, cuisine types, or specific menu items, prioritizing relevant results dynamically.

9 Conclusion

The GetHere food delivery platform successfully demonstrates comprehensive relational database implementation with:

- **9 Database Tables:** User, Restaurant, Restaurant_Manager, Courier, Food, Menu, Orders, Positions, Task
- **Complete CRUD Operations:** All entities support Create, Read, Update, Delete with validation
- **Complex SQL Queries:** Multi-table JOINs (5 tables), GROUP BY aggregations, nested subqueries, window functions
- **Data Integrity:** Foreign key constraints, CHECK constraints, CASCADE/SET NULL/RESTRICT actions
- **Business Logic:** Trust-weighted ratings, eligibility-based job applications, position-based tracking

9.1 Future Improvements

- Real-time delivery tracking with geolocation
- Payment gateway integration
- Push notifications for order status
- Mobile application development

10 References

1. MySQL 8.0 Reference Manual: `https://dev.mysql.com/doc/refman/8.0/en/`
2. Flask Documentation: `https://flask.palletsprojects.com/`
3. Bootstrap 5 Documentation: `https://getbootstrap.com/docs/5.0/`